

Deployment

Die letzte, finale Phase im CRISP-DM-Prozess ist die Operationalisierung des Modells, auch Deployment genannt. Hierzu wird das vorher erstellte Modell in die IT-Infrastruktur integriert, so dass es durchgehend in Betrieb ist. Was dabei nicht vergessen werden darf, ist die kontinuierliche Überwachung einerseits der Verfügbarkeit, andererseits der Performance des Modells. Ziel dieser Phase war es, den zuvor entwickelte Prototyp im Unternehmen bereitzustellen.

Zielform und Bereitstellung

Als Zielform wurde die SAP Cloud Foundry (CF) gewählt. Als Runtime innerhalb der BTP ermöglicht sie eine einfache Bereitstellung direkt aus dem Quellcode, ohne dass Server verwaltet werden müssen. Die bereits im Modeling beschriebene Systemarchitektur, bei der die Streamlit-App als Frontend dient, wurde auf dieser Plattform abgebildet.

Die technische Bereitstellung des Python-Projekts erfolgte über den Standard-`cf push`-Befehl. Gesteuert wurde dieser Prozess durch zwei wesentliche Dateien:

- *manifest.yml*: Diese Konfigurationsdatei definiert die Rahmenbedingungen für die Anwendung. Sie legt den Namen (`sap-cap-pipeline-generator`), den Speicher (512M) und den `python_buildpack` fest. Der entscheidende Eintrag ist der `command`, der Streamlit anweist, auf dem dynamisch von CF zugewiesenen `$PORT` zu lauschen.
- *requirements.txt*: Enthält alle Python-Abhängigkeiten (wie `streamlit` und `requests`), die der Buildpack automatisch bei der Bereitstellung installiert.

Konfigurations- und Sicherheitsmanagement

Ein zentraler Aspekt der Operationalisierung ist die strikte Trennung von Code und Konfiguration. Sensible Daten, insbesondere die Credentials für den SAP AI Core und XSUAA-Endpunkte, dürfen nicht im Quellcode gespeichert werden.

Lokale Konfigurationsdateien (wie die *.env*) wurden daher über die *.cfignore*-Datei explizit von der Bereitstellung ausgeschlossen.

Alle für den Betrieb notwendigen Credentials und Endpunkte wurden stattdessen nach dem Deployment als Cloud Foundry Environment Variables (Umgebungsvariablen) gesetzt. Dies geschah über den `cf set-env`-Befehl. Dieser Ansatz stellt sicher, dass der Code sicher im Git-Repository gespeichert werden kann und Konfigurationen flexibel im Betrieb geändert werden können.

Überwachung im Betrieb

Um die von CRISP-DM geforderte kontinuierliche Überwachung sicherzustellen, wurden die folgenden nativen Werkzeuge der Cloud Foundry genutzt:

- Die Verfügbarkeit (Status der App) wurde mittels `cf apps` überwacht.
- Die Performance und Fehleranalyse (z.B. bei der API-Kommunikation mit AI Core) erfolgte durch die Analyse der Echtzeit-Logs mittels `cf logs sap-cap-pipeline-generator --recent`.

Nach einem finalen Neustart durch den Befehl `cf restart`, um die Umgebungsvariablen zu laden, war der Prototyp erfolgreich operationalisiert und für die Zielgruppe erreichbar.